САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Малахов Алексей

К3439

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Привязать сайт из ЛР1 к внешнему API средствами `fetch`/`axios`/`xhr`. Реализовать моковое API средствами JSON Server и подключить к нему авторизацию, как в примерах темы «Имитация работы с API». Вариант 5: сервис для обмена рецептами и кулинарных блогов.

Ход работы

Моковое API

Сайт из ЛР1 скопирован в `lab2/public/` (папка `lab1` осталась без изменений) и раздаётся тем же сервером, что и API. Как в примере `18_bootstrap_api_example`, использованы JSON Server и middleware json-server-auth. Версии зафиксированы точно, потому что json-server-auth работает только с JSON Server ветки 0.17:

```
"devDependencies": {
  "json-server": "0.17.4",
  "json-server-auth": "2.1.0"
}
```

Команда `npm start` запускает сервер так:

```
json-server db.json \
  -m ./server/missing.js ./node_modules/json-server-auth ./server/guards.js \
  -r routes.json --static ./public --port 3000
```

Данные из `js/data.js` первой лабораторной перенесены в `scripts/seed-data.js`. Скрипт `reset-db.js` собирает из них `db.json` и хеширует пароли демо-пользователей через bcrypt, поэтому открытых паролей в базе нет. В базе 7 коллекций:

Ресурс	Права	Назначение
`users`	600	аккаунты: email, хеш пароля, имя
`profiles`	644	публичные профили кулинаров
`recipes`	644	рецепты с ингредиентами, шагами, видео
`comments`	644	комментарии к рецептам
`likes`	644	лайки: `recipeId` и `userId`
`follows`	644	подписки: кто (`userId`) на кого (`authorId`)
`saved`	640	закладки пользователя

Права задаются в `routes.json`. Первая цифра означает права владельца, вторая вошедшего пользователя, третья гостя (6: чтение и запись, 4: только чтение). Например, `"/recipes*": "/644/recipes$1"`: рецепты читают все, а создавать,

менять и удалять может только автор. Владельца json-server-auth определяет по полю `userId`, поэтому все пользовательские записи содержат это поле.

У json-server-auth нашлись дыры, поэтому после него подключён свой небольшой middleware `server/guards.js`. Для `PUT` json-server-auth сверяет `userId` только в теле запроса, а не у существующей записи, и через `PUT` можно было перезаписать чужой рецепт. Клиент меняет данные только через `PATCH`, поэтому `PUT` запрещён целиком (405). Через `PATCH` нельзя сменить владельца записи, а лайки, закладки и подписки нельзя менять вовсе (403): клиент их только создаёт и удаляет. Повторный лайк, закладка или подписка от того же пользователя отклоняются (409). Проверка и запись выполняются в одном синхронном обработчике, поэтому даже одновременные запросы не создают дублей:

```
if (req.method === 'POST' && keys && req.app.db) {
  const collection = req.app.db.get(resource)
  const exists = collection
    .some((item) => keys.every((key) => String(item[key]) === String(req.body[key])))
    .value()
  if (exists) {
    res.status(409).jsonp('Already exists')
    return
  }
  const item = collection.insert(req.body).value()
  req.app.db.write()
  res.status(201).jsonp(item)
  return
}
```

Ещё один middleware, `server/missing.js`, подключён перед json-server-auth: на `PATCH` или `DELETE` несуществующей записи он сразу отвечает 404, потому что json-server-auth в этом случае падает с ошибкой и отвечает 401. Если запись уже удалили в другой вкладке, клиент на ответ 404 или 409 перезапрашивает актуальное состояние и перерисовывает кнопку.

Работа с API на клиенте

Все запросы проходят через функцию `api()` из `js/api.js`. Она добавляет заголовок `Authorization` с токеном, превращает ответ в JSON и переводит ошибки сервера в понятный текст:

```
async function api(path, { method = 'GET', body } = {}) {
  const headers = {}
  const token = getToken()
  if (body) headers['Content-Type'] = 'application/json'
  if (token) headers.Authorization = `Bearer ${token}`

  let response
  try {
    response = await fetch(API_URL + path, {
      method,
      headers,
      body: body && JSON.stringify(body),
    })
  } catch {
    throw new ApiError(0, 'Сервер недоступен. Запустите API командой npm start.')
  }

  const data = await response.json().catch(() => null)
  if (!response.ok) {
    if (response.status === 401 && token) {
      logout()
      location.href = loginUrl()
    }
    const text = typeof data === 'string' ? data : ''
    const message = ERROR_TEXT[text] || `Ошибка сервера (${response.status})`
    throw new ApiError(response.status, message)
  }
  return data
}
```

Главная загружает рецепты вместе с лайками одним запросом `GET /recipes?_embed=likes` и сортирует их по числу лайков (Рис. 1).

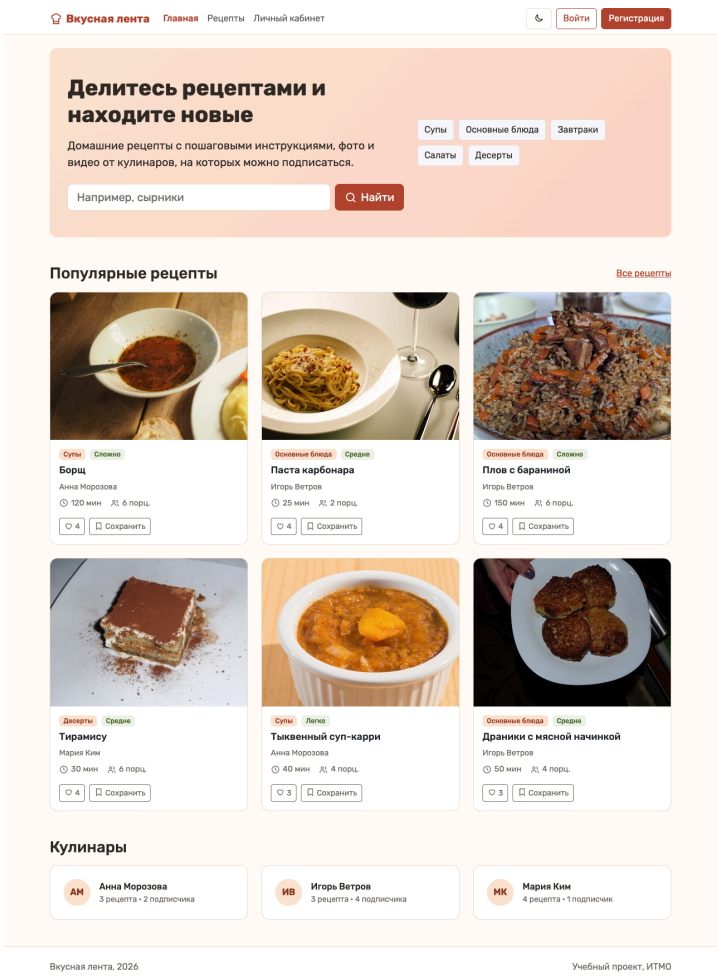


Рис. 1. Главная: популярные рецепты и кулинары из API

Поиск

Фильтры по типу блюда, сложности, времени и названию передаются серверу как параметры запроса JSON Server: `type`, `difficulty` (можно несколько), `time_lte`, `title_like`. Например, для завтраков лёгкой сложности уходит `GET /recipes?type=Завтраки&difficulty=Легко&_embed=likes`. Условие «есть все отмеченные ингредиенты» JSON Server выразить не умеет, поэтому оно проверяется на клиенте по полю `tags`. Чтобы ответ на старый запрос не перезаписал более новый, у каждого запроса есть номер, и устаревшие ответы отбрасываются. Спецсимволы в строке поиска экранируются, потому что `title_like` воспринимается сервером как регулярное выражение.

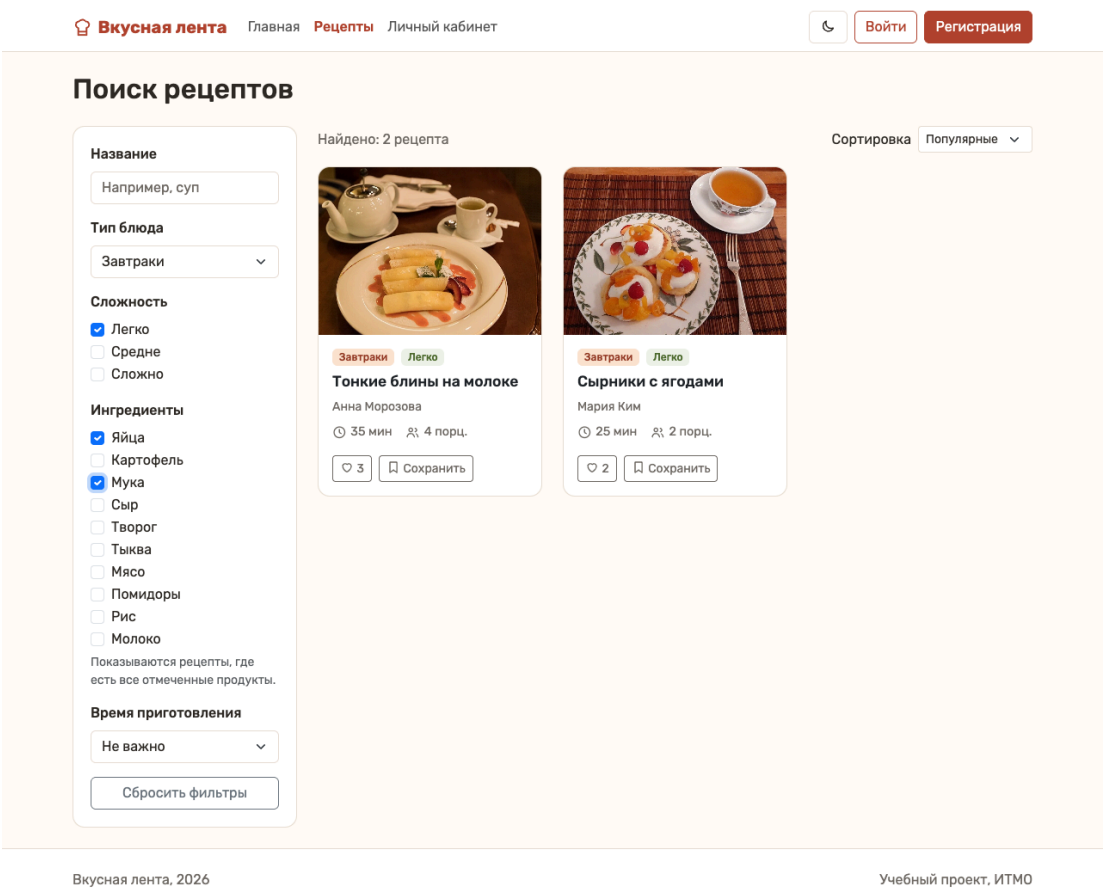


Рис. 2. Поиск: фильтры отправляются в запросе к API

Авторизация

Вход отправляет `POST /login`, регистрация `POST /register`. В ответе приходит `accessToken` (JWT на 1 час) и данные пользователя. В `localStorage` сохраняются только токен и пользователь, пароль не хранится. Перед каждым запросом срок действия токена проверяется по полю `exp`, просроченный токен удаляется. После регистрации создаётся публичный профиль `POST /profiles`.

Если сервер отклонил токен (например, у него испорчена подпись), функция `api()` получает 401, удаляет сессию и переводит на страницу входа. Неверный пароль возвращает 400 и сообщение «Неверный email или пароль» (Рис. 3), а регистрация с занятым email возвращает «Пользователь с таким email уже зарегистрирован». После успешной регистрации сразу открывается личный кабинет нового пользователя (Рис. 4). Личный кабинет без входа открыть нельзя: `requireAuth()` переводит на `login.html?next=profile.html`, а после входа пользователь возвращается на исходную страницу. Лайк, закладка, подписка или комментарий от гостя тоже ведут на вход.

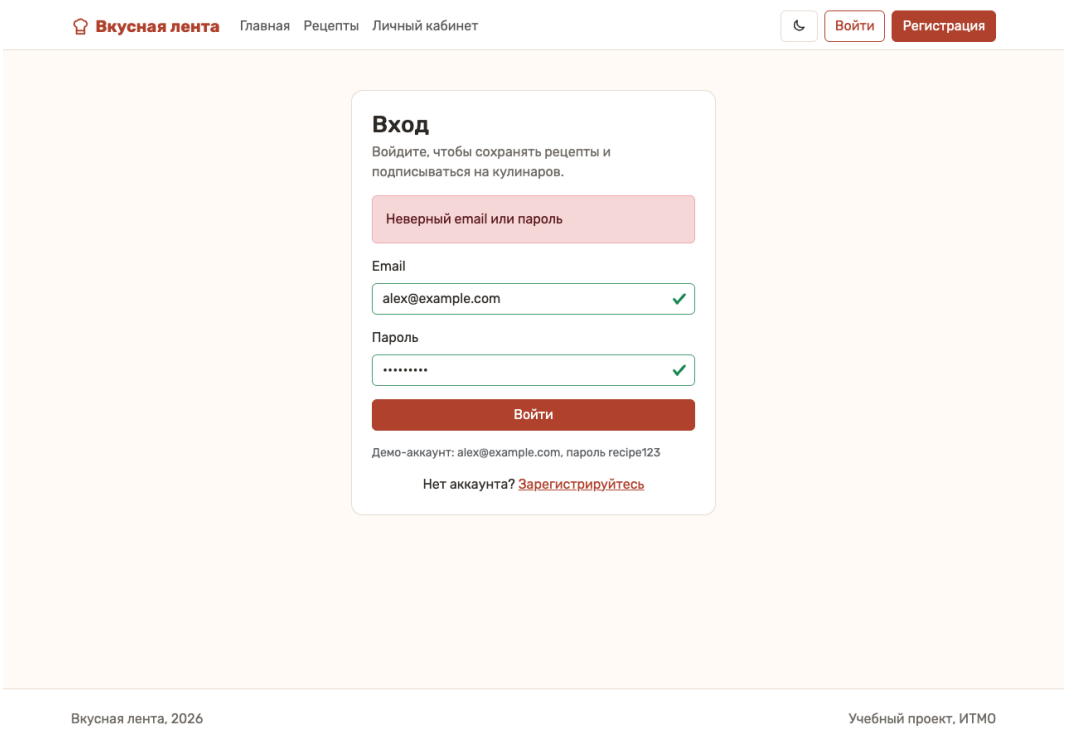


Рис. 3. Ответ сервера на неверный пароль

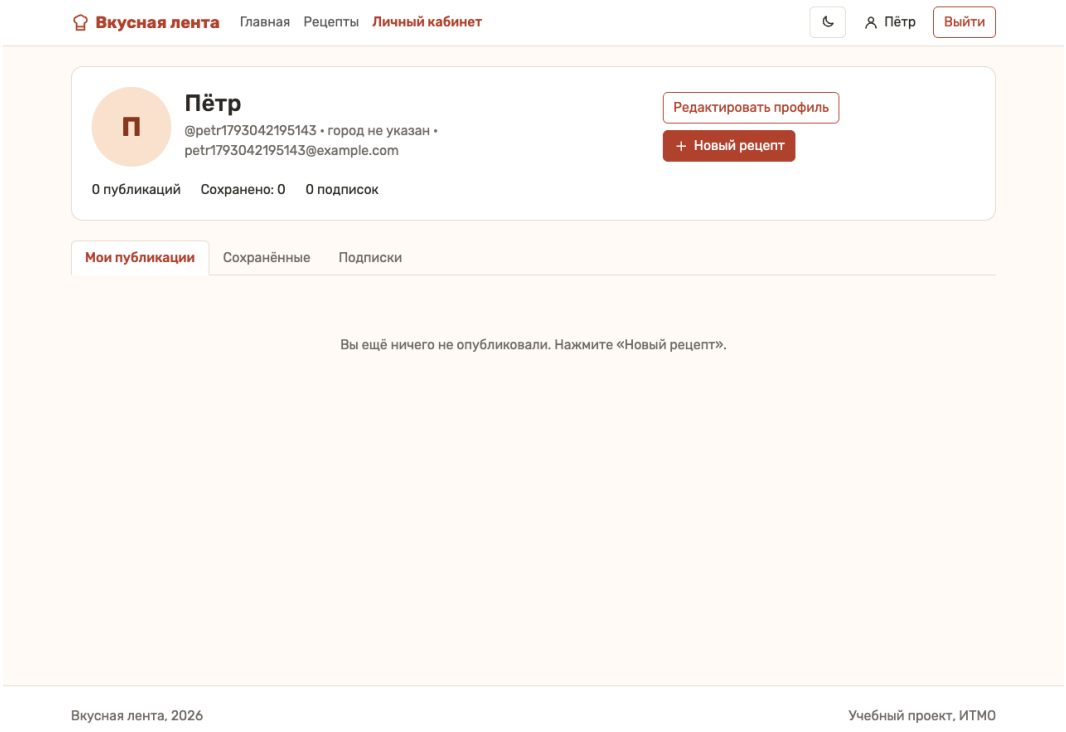


Рис. 4. Кабинет нового пользователя сразу после регистрации

Социальные функции и личный кабинет

Лайк создаётся запросом `POST /likes` с телом `{ recipeId, userId }` и снимается `DELETE /likes/:id`. Так же устроены закладки (`/saved`) и подписки (`/follows`). Комментарий отправляется `POST /comments`. После перезагрузки страницы всё сохраняется, потому что данные лежат на сервере (Рис. 5).

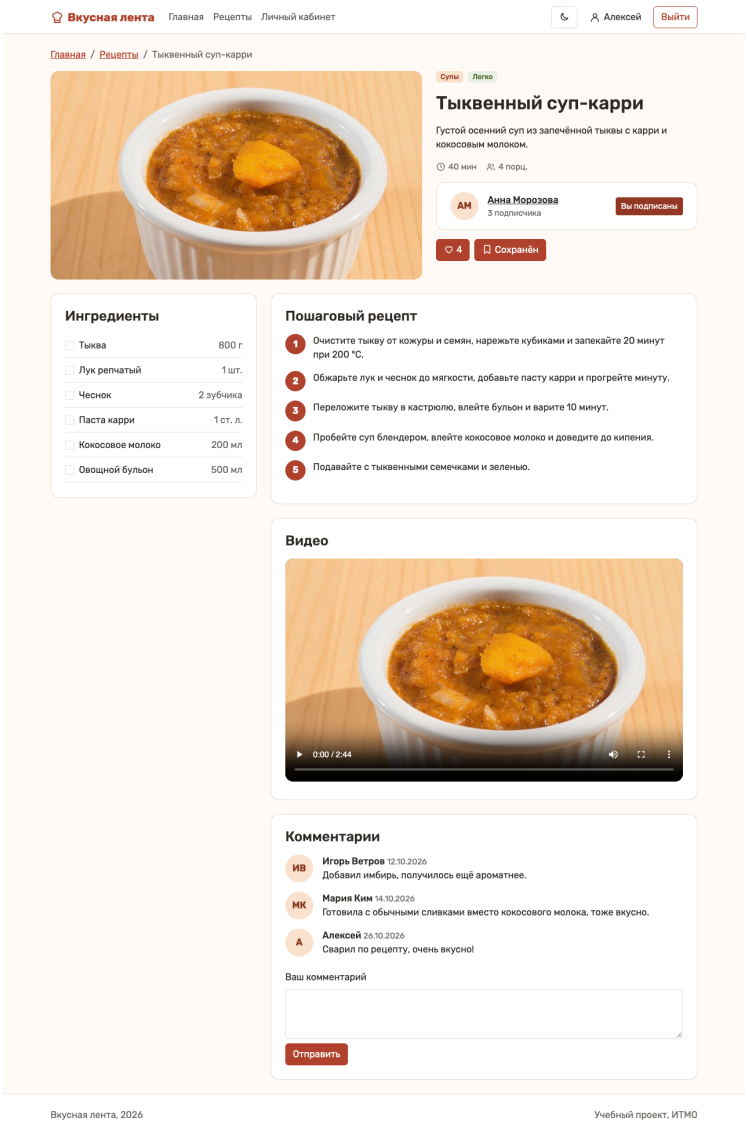


Рис. 5. Рецепт после лайка, сохранения, подписки и комментария

В личном кабинете (Рис. 6) мои рецепты загружаются запросом `GET /recipes?userId=4`, сохранённые по `id` из закладок, подписки из `/follows`. Новый рецепт создаётся `POST /recipes`, свой рецепт можно удалить, профиль меняется через `PATCH /profiles/4`.

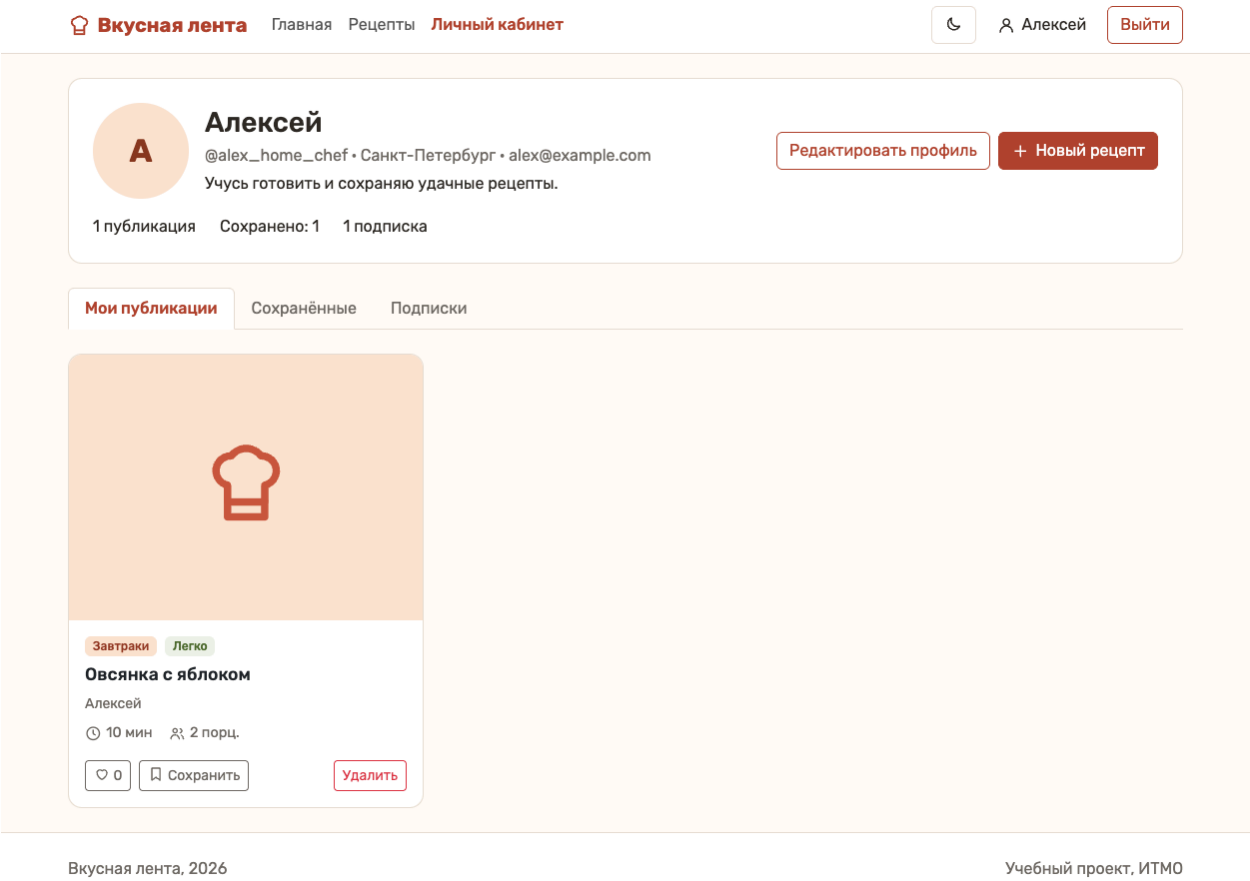


Рис. 6. Личный кабинет после публикации рецепта

Проверка доступа на сервере

Скрытая кнопка в интерфейсе не защищает данные, поэтому права проверялись прямыми запросами с токеном пользователя Алексей (`id = 4`). Также проверено, что название рецепта с кавычками и атрибутом `onload` выводится как текст: функция `escapeHtml` экранирует не только `<` и `>`, но и кавычки, поэтому строка не может выйти за пределы атрибута.

Запрос	Ответ
`DELETE /recipes/1` (рецепт Анны)	403
`POST /likes` с `userId: 1` (лайк от чужого имени)	403
`GET /users` без токена	401
`POST /login` с неверным паролем	400
`POST /register` с занятым email	400
`PUT /recipes/1` (полная замена записи)	405
`PATCH /profiles/4` со сменой `userId`	403
повторный `POST /likes` того же пользователя	409
`PATCH /likes/:id` (подмена рецепта у лайка)	403
`DELETE /likes/99999` (записи нет)	404

У json-server-auth есть ещё одно ограничение. При чтении списка с правами 600 он проверяет только, есть ли в коллекции хотя бы одна запись текущего пользователя. У нового пользователя без закладок запрос `GET /saved?userId=…` поэтому получал бы 403, а закладки запрашиваются на каждой странице, и сайт перестал бы работать. Поэтому у `saved` права 640: читать закладки могут только вошедшие пользователи (и, к сожалению, не только свои), а создавать и удалять только владелец. Для учебного мок-сервера это допустимо, но в настоящем бэкенде список нужно фильтровать на сервере.

Если сервер перестал отвечать, вместо пустой страницы выводится сообщение «Сервер недоступен» (Рис. 7). Сайт раздаётся тем же сервером, поэтому при полностью остановленном сервере новую страницу браузер не откроет вовсе, а сообщение появится на уже открытой.

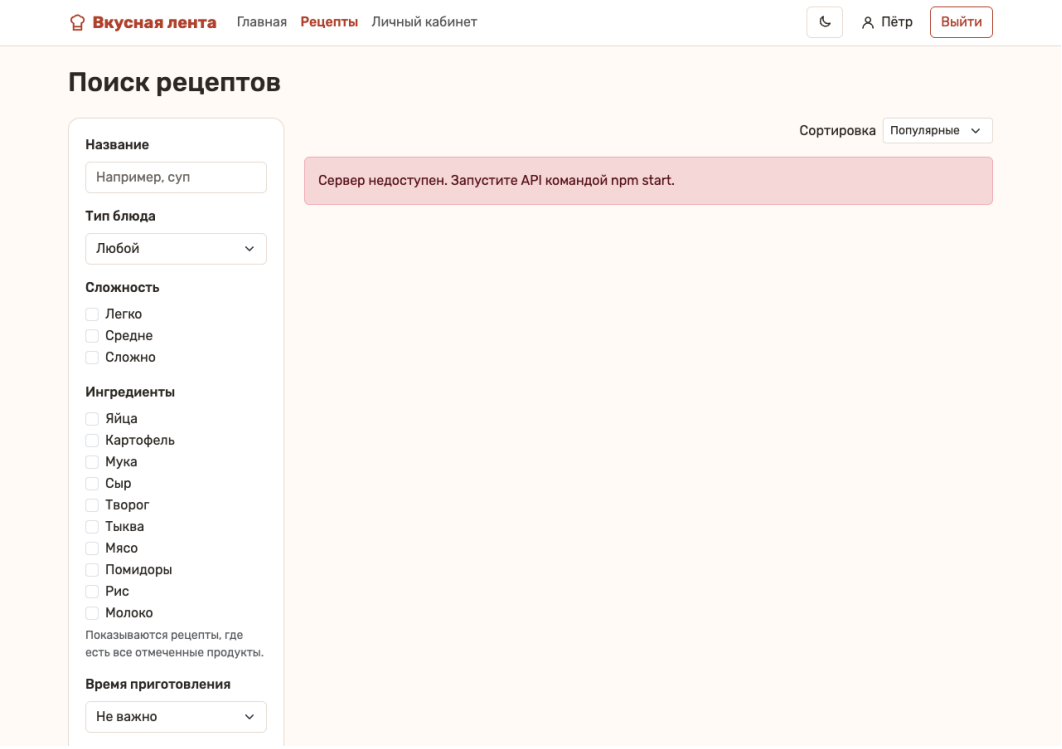


Рис. 7. Сообщение при недоступном API

Проверка

Сервер запускался командой `npm start`, база перед проверкой пересоздавалась `npm run reset-db`. Сценарии проходились в Google Chrome под macOS (через Playwright), всего 21: загрузка главной и поиска из API, фильтры в запросе, спецсимволы в поиске, переход гостя на вход, неверный и верный вход, лайк, закладка, подписка и комментарий с проверкой после перезагрузки, запреты сервера из таблицы выше, название рецепта с HTML-атрибутом, создание и удаление рецепта, испорченный токен, изменение профиля, отписка, выход, защита кабинета, просроченный токен, регистрация с занятым и новым email,

недоступный сервер, отсутствие горизонтальной прокрутки на ширине 390 px. Ошибок JavaScript не было.

Вывод

Сайт рецептов работает с моковым API на JSON Server: все данные (рецепты, профили, комментарии, лайки, закладки, подписки) загружаются и сохраняются через `fetch`. Авторизация сделана на json-server-auth: вход и регистрация выдают JWT, токен передаётся в заголовке `Authorization`, а права на запись проверяет сервер по полю `userId`. Обработаны ошибки входа, истёкший токен и недоступный сервер.